

Project 2: SnailSaver 多核性能优化与碰撞检测 Write-up

一、性能分析与优化点定位

在着手优化之前，首先需要明确程序的性能瓶颈。由于初始的屏幕保护程序采用了暴力双重循环来进行线段碰撞检测，其时间复杂度高达 $O(N^2)$ ，导致帧率极低。

分析过程：

在 1000 帧的条件下，使用MacOS上的sample工具运行程序，对程序运行过程中的具体情况进行捕捉，运行结果如下：

```
1 Call graph:
2   4230 Thread_34250832   DispatchQueue_1: com.apple.main-thread   (serial)
3   4230 start   (in dyld) + 6076   [0x19c7e2b98]
4   4230 main   (in screensaver) + 424   [0x1045e9b34]
5   4230 LineDemo_update   (in screensaver) + 36   [0x1045e994c]
6   4229 CollisionWorld_updateLines   (in screensaver) + 20   [0x1045e87fc]
7   + 4023 CollisionWorld_detectIntersection   (in screensaver) + 160
   [0x1045e89b0]
8   + ! 1441 intersect   (in screensaver) + 368,320,...
   [0x1045e9090,0x1045e9060,...]
9   + ! 948 intersect   (in screensaver) + 392   [0x1045e90a8]
10  + ! : 948 intersectLines   (in screensaver) + 96,88,...
   [0x1045e92f8,0x1045e92f0,...]
11  + ! 563 intersect   (in screensaver) + 452   [0x1045e90e4]
12  + ! : 563 intersectLines   (in screensaver) + 112,144,...
   [0x1045e9308,0x1045e9328,...]
13  + ! 475 intersect   (in screensaver) + 260   [0x1045e9024]
14  + ! : 475 intersectLines   (in screensaver) + 84,0,...
   [0x1045e92ec,0x1045e9298,...]
15  + ! 372 intersect   (in screensaver) + 336   [0x1045e9070]
16  + ! : 372 intersectLines   (in screensaver) + 84,288,...
   [0x1045e92ec,0x1045e93b8,...]
17  + ! 93 intersect   (in screensaver) + 188   [0x1045e8fdc]
18  + ! : 93 Vec_multiply   (in screensaver) + 0   [0x1045e9e30]
19  + ! 37 intersect   (in screensaver) + 204   [0x1045e8fec]
20  + ! : 37 Vec_add   (in screensaver) + 0   [0x1045e9e50]
21  + ! 31 intersect   (in screensaver) + 168   [0x1045e8fc8]
22  + ! : 31 Vec_add   (in screensaver) + 0   [0x1045e9e50]
23  + ! 24 intersect   (in screensaver) + 136   [0x1045e8fa8]
24  + ! : 24 Vec_subtract   (in screensaver) + 0,4
   [0x1045e9c7c,0x1045e9c80]
25  + ! 21 intersect   (in screensaver) + 152   [0x1045e8fb8]
26  + ! : 21 Vec_multiply   (in screensaver) + 0   [0x1045e9e30]
27  + ! 13 intersect   (in screensaver) + 80   [0x1045e8f70]
28  + ! : 13 Vec_makeFromLine   (in screensaver) + 8   [0x1045e9c74]
```

```

29      + ! 5 intersect (in screensaver) + 120 [0x1045e8f98]
30      + ! 5 Vec_makeFromLine (in screensaver) + 4 [0x1045e9c70]
31      + 115 intersect (in screensaver) + 876,872, ...
[0x1045e928c,0x1045e9288, ...]
32      + 91 CollisionWorld_detectIntersection (in screensaver) +
104,144, ... [0x1045e8978,0x1045e89a0, ...]
33      1 CollisionWorld_updateLines (in screensaver) + 28 [0x1045e8804]
34      1 CollisionWorld_updatePosition (in screensaver) + 60
[0x1045e8ab0]
35      1 Vec_multiply (in screensaver) + 0 [0x1045e9e30]

```

Total number in stack (recursive counted multiple, when ≥ 5):

Sort by top of stack, same collapsed (when ≥ 5):

intersectLines (in screensaver)	2358	
intersect (in screensaver)	1556	
Vec_multiply (in screensaver)	115	
CollisionWorld_detectIntersection (in screensaver)		91
Vec_add (in screensaver)	68	
Vec_subtract (in screensaver)	24	
Vec_makeFromLine (in screensaver)	18	

分析结果：

1. 通过日志底部的 `Sort by top of stack` 区域可以发现：

- `intersectLines` 占用了 2358 次采样。
- `intersect` 占用了 1556 次采样。

这两个函数加起来消耗了程序 90% 以上的执行时间。 `CollisionWorld_detectIntersection` 在不断循环调用这些底层几何计算。

2. Call graph 的最顶层 `4230 Thread_34250832 DispatchQueue_1: com.apple.main-thread (serial)` 说明：

所有的 4230 次采样全部集中在主线程上，且被标记为串行，没有使用并行计算。

优化思路确定：

针对分析结果，可以将优化分为两个方向：

1. 使用四叉树进行算法层面的空间降维
2. 使用 `Opencilk` 使程序能够多核并发运行

在test部分中，我们先进行了算法层面的优化。

二、改进思路与伪代码实现

四叉树算法优化

数据结构设计：

1. 边界记录：存储节点物理边界坐标及树深度。

2. 扁平存储：使用连续内存数组存放线段 ID，提升 Cache 命中率。
3. 动态扩容：支持 1.5 倍自动扩容，减少内存分配开销。

关键算法逻辑：

1. 边界预测：利用 0.5 步长预计算扫掠包围盒，确保判定物理准确。
2. 动态分裂：节点容量超限且未达深度上限时，触发分裂。

算法工程优化：

1. 结构扁平化：用数组代替链表
2. 跨界唯一性：跨界线段滞留父节点
3. AABB 预判：扫掠包围盒 $O(1)$ 粗筛，减少无效几何运算

四叉树构建与查询伪代码：

```
1 // 1. 定义四叉树节点
2 struct QuadtreeNode {
3     Box bounds;           // 当前节点的物理边界
4     List<LineID> items;    // 包含的线段 ID
5     QuadtreeNode* children[4]; // 四个子节点
6 }
7
8 // 2. 将所有线段插入四叉树（针对每帧）
9 function build_tree(lines, num_lines):
10     root = create_node(WORLD_BOUNDS)
11     for i from 0 to num_lines - 1:
12         insert_line(root, lines[i])
13
14 function insert_line(node, line):
15     if node has children and line fits completely in a child:
16         insert_line(node.children[target_child], line)
17     else:
18         node.items.append(line.id)
19         // 若节点容量超限且未达最大深度，则分裂
20         if node.items.size > CAPACITY and node.depth < MAX_DEPTH:
21             split_node(node)
22
23 // 3. 收集潜在的碰撞对 (Candidate Pairs)
24 function collect_pairs(node, active_list, pair_list):
25     // 当前节点内的线段与 active_list 中的线段进行包围盒粗筛
26     for item in node.items:
27         for active in active_list:
28             if bounds_overlap(item, active):
29                 pair_list.append((item, active))
30
31     // 当前节点内部的线段互相检测
32     for i, j in node.items (all pairs):
33         if bounds_overlap(i, j):
34             pair_list.append((i, j))
```

```
35
36     // 递归子节点
37     next_active = active_list + node.items
38     for child in node.children:
39         collect_pairs(child, next_active, pair_list)
```

三、编程过程中遇到的 Bug 与陷阱

四叉树过度分裂

- **现象：**
为了追求最少量的 `intersect` 调用，我将四叉树的最大深度设置为 10 层。结果整体运行时间反而变慢了。
- **原因：**
分配/销毁极大量的四叉树节点、以及在过深的树中进行内存寻址的开销，远远超过了节省下来的那点碰撞计算开销。
- **解决：**
通过测试调整，找到了一个甜点值，最终将 `QT_MAX_DEPTH` 设定为 6，节点容量上限 `QT_NODE_CAPACITY` 设为 12，达到了时空开销的最佳平衡。

四、尝试过但无显著提升的“无用功”

手动循环展开

尝试：

实验中我尝试将碰撞检测中的小循环手动展开，试图减少跳转指令。

结果：

但是这牺牲了代码的可读性和维护性，且现代编译器的优化器能自动执行更高效的循环展开，手动优化不仅没提速，反而可能造成指令缓存溢出。

五、性能对比

原版：

```
1  › ./screensaver 4000
2  Number of frames = 4000
3  Input file path is: input/mit.in
4  ---- RESULTS ----
5  Elapsed execution time: 40.345394s
6  1262 Line-Wall Collisions
7  19806 Line-Line Collisions
8  ---- END RESULTS ----
```

测试版优化后：

```
1  › ./screensaver 4000
2  Number of frames = 4000
3  Input file path is: input/mit.in
4  ---- RESULTS ----
5  Elapsed execution time: 38.700500s
6  1262 Line-Wall Collisions
7  19806 Line-Line Collisions
8  ---- END RESULTS ----
```